



Software Requirements Specification

Final As-Built Specification

THE LAUNCHPAD / FINTECH BRIEF 03/10

Not Your College Internship

LIVE APPLICATION
split-pe-neon.vercel.app

GITHUB REPOSITORY
github.com/Aaryan0612/SplitPe

PREPARED BY

Aaryan Kuchekar

SUBMISSION DATE

17 July 2026

VERSION

1.0

Document Control

Field	Value
Document title	SPLITPE Software Requirements Specification
Version	1.0
Status	Final As-Built Specification
Author	Aaryan Kuchekar
Date	17 July 2026
Project category	Mobile-first fintech prototype
Internship context	The Launchpad - Not Your College Internship - Fintech Brief 03/10
Live URL	https://split-pe-neon.vercel.app/
Repository URL	https://github.com/Aaryan0612/SplitPe

Status Classification

This document uses four implementation classifications:

- **Implemented:** present in the completed application and supported by source or verification evidence.
- **Partially implemented:** present with a material difference from the original requirement.
- **Deferred:** intentionally excluded from the delivered version and retained as future scope.
- **Not applicable:** explicitly outside the prototype boundary or unnecessary for the delivered architecture.

1. Executive Summary

SPLITPE is a polished, mobile-first expense-splitting experience for students living in shared flats and paying guest accommodation. It enables a user to describe a shared bill, edit the participating flatmates, select the payer, and immediately see an equal-share settlement summary written in plain language.

The delivered product is a single-page React application and a client-side prototype. Its central value proposition is clarity: **Split expenses. Not friendships.** It does not process payments, store household accounts, collect banking credentials, or depend on a backend.

The implementation includes a responsive landing experience, equal, custom-amount, and itemised calculations, deterministic paise-level allocation, inline validation, explicit who-pays-whom results, copyable summaries, browser-local expense history, honest privacy language, a custom SPLITPE brand mark, and a verified Vercel deployment.

2. Problem Statement

Students sharing flats and PGs routinely pay for groceries, utilities, food, rent add-ons, and household supplies on behalf of one another. These small obligations are easily lost across memory and chat messages. By the time a household settles, unclear totals and repeated reminders can make a straightforward repayment feel socially awkward.

Many existing tools expose more accounting complexity than a quick household split requires. The product opportunity is therefore a lightweight experience that communicates the amount, payer, equal share, and settlement direction without requiring an account or financial integration.

3. Product Objectives

3.1 Objectives and Success Criteria

ID	Objective	Success criterion	Status
OBJ-01	Explain the product immediately	A phone visitor can identify the audience, problem, and primary action above the fold	Implemented
OBJ-02	Demonstrate the Add - Split - Settle flow	Three ordered steps use concise, consistent copy	Implemented
OBJ-03	Produce correct shared-expense splits	Equal, custom, and itemised shares sum exactly to the entered total	Implemented
OBJ-04	Make settlement direction explicit	One plain-language sentence is shown for every non-payer	Implemented
OBJ-05	Prevent misleading output	Invalid amount or participant data replaces the result with a correction state	Implemented
OBJ-06	Work across mobile and desktop	No horizontal overflow at 320, 390, or 1440 pixel verification widths	Implemented
OBJ-07	Communicate prototype boundaries	Trust copy states that no payments are processed and no bank details are collected	Implemented
OBJ-08	Deploy without operational dependencies	Static production build and live Vercel URL require no environment variables	Implemented

3.2 Non-Goals

- Real UPI transfers, payment collection, wallet functionality, or transaction status.
- Authentication, user profiles, household membership, invitations, or cross-device sync.
- Backend APIs, databases, analytics, trackers, or cloud storage.
- Weighted or percentage-based split modes.
- Cross-device expense history, recurring bills, reminders, notifications, or administrative dashboards.
- Claims of bank-grade encryption, regulatory approval, or financial institution status.

These items are **Not applicable** to the delivered prototype and are not represented as completed features.

4. Target Users and User Needs

4.1 Primary Persona

A college student living with two to five flatmates who occasionally pays a shared household bill and wants a correct settlement summary in under a minute.

4.2 Context of Use

- Primarily mobile, often immediately after a shared purchase.
- Short, focused interaction with no onboarding.
- A need to explain the split to other participants in familiar language.
- Limited tolerance for account creation or financial credential requests.

4.3 User Needs

Need	Product response
Understand what the product does quickly	Direct hero message and believable default split preview
Enter a shared expense with minimal effort	Pre-filled calculator with editable bill and participant data
Identify the person who paid	Conventional payer select populated from current participants
Know exactly who owes whom	Explicit settlement sentences for each non-payer
Trust the prototype boundary	Visible no-bank-details and no-payment-processing language
Use the calculator comfortably on a phone	Mobile-first stacking, 20 pixel gutters, and 44 pixel minimum targets

5. Project Scope

5.1 Implemented MVP

- Sticky minimal header with wordmark, section anchor, and calculator CTA.
- Hero with audience cue, value proposition, supporting copy, primary CTA, secondary anchor, microcopy, and example split preview.
- Ordered Add - Split - Settle explanation.
- Trust section with UPI settlement intent and honest prototype language.
- Final CTA and minimal footer.
- Responsive single-page presentation deployed to one live Vercel URL.

5.2 Implemented Beyond-MVP Functionality

- Live equal-split calculator with editable expense, amount, participants, and payer.
- Two to six participants with safe add and remove boundaries.
- Instant client-side recalculation.
- Inline amount and participant validation.
- Deterministic integer-paise remainder handling.
- Invalid-result state that suppresses misleading settlement values.
- Custom-amount splitting with exact-total validation.
- Itemised splitting with per-item participant selection.
- Clipboard-ready settlement summaries with success or failure feedback.
- Up to 20 saved expense snapshots in browser localStorage, with restore, copy, and delete controls.
- Reduced-motion behaviour.
- Custom vector SPLITPE brand mark and matching SVG favicon.

5.3 Deferred Functionality

Original item	Status	As-built decision
Web Share API	Deferred	Clipboard copy is implemented; native share-sheet integration is not
Authentication and shared households	Deferred	Future production scope
Cross-device expense history	Deferred	Current history is device-local and intentionally requires no account
UPI settlement intent/deep link	Deferred	The app describes settlement intent but does not initiate payment
Reminders and notifications	Deferred	Future household workflow
Offline/PWA support	Deferred	Static web delivery only

5.4 Explicit Non-Goals

Authentication, databases, payment APIs, routing, analytics, dashboards, dark mode, and regulatory or encryption claims are **Not applicable** to this version.

5.5 As-Built Changes

1. The final header uses a custom equal-share vector mark instead of the originally suggested small rupee mark. The mark is implemented in BrandMark.jsx and mirrored in public/favicon.svg.
2. The amount parser accepts any positive value representable in paise, starting at ₹0.01. The original SRS stated a ₹1 minimum, so that specific lower-bound requirement is **Partially implemented**.
3. Clipboard copy was added for both the active result and saved history entries; native Web Share remains deferred.
4. A schema-checked localStorage history now retains up to 20 complete calculator snapshots and supports restore and deletion.
5. Custom-amount and itemised split modes were added while retaining equal split as the default.
6. Reduced-motion support, originally listed as optional P2, was implemented in the final CSS.
7. The finished application includes an SVG favicon and reusable brand component beyond the original component list.
8. Focused Node tests now cover paise allocation, custom totals, itemised allocation, share text, and history storage safety.

6. User Stories

ID	User story	Acceptance outcome	Status
US-01	As a first-time visitor, I want to understand SPLITPE immediately so I can judge its relevance	Hero communicates audience, problem, and CTA	Implemented
US-02	As a flatmate, I want to enter a shared amount and participant names so I can calculate each share	Editable bill, amount, and 2-6 participant controls	Implemented
US-03	As the person who paid, I want to select myself as payer so I can see who owes me	Paid-by select updates result and settlement direction	Implemented
US-04	As a participant, I want plain-language results so I do not have to interpret a ledger	Each non-payer receives an explicit settlement sentence	Implemented
US-05	As a cautious user, I want to know whether bank details are collected or payments are processed	Trust section states both boundaries directly	Implemented
US-06	As a mobile user, I want large controls and a vertical flow without sideways scrolling	Live checks confirm 44 pixel targets and no horizontal overflow at 320 and 390 pixels	Implemented
US-07	As a returning user, I want my previous split restored	Saved browser-local history survives refresh and can restore a snapshot	Implemented
US-08	As a user, I want to copy the settlement summary	Copy controls provide success or failure feedback	Implemented

7. Information Architecture

The delivered application uses one route and a conventional vertical flow:

Hero -> How it Works -> Live Calculator -> Trust Section -> Final CTA / Footer

1. **Sticky header:** SPLITPE brand link, How it works anchor, and Try Splitpe CTA.
2. **Hero:** value proposition and a complete example split preview.
3. **How it Works:** three connected steps explaining Add, Split, and Settle.
4. **Live Calculator:** bill details, people, paid-by selection, and emphasized result.
5. **Trust Section:** prototype, credential, and browser-state boundaries.
6. **Final CTA/Footer:** returns the user to the calculator and closes the page.

Navigation uses native anchors for the top, how-it-works, and calculator sections. No router or secondary page is present.

8. Functional Requirements

FR-01 Navigation and Section Scrolling

- **Description:** Provide conventional in-page navigation between the header, explanation, calculator, and top of page.
- **Inputs:** Header and hero links using hash targets.
- **Expected behaviour:** Calculator CTAs target #calculator; explanation links target #how-it-works; the wordmark targets #top.
- **Validation and edge cases:** Native anchor navigation remains usable without JavaScript routing. Reduced-motion preferences disable smooth scrolling.
- **Actual status:** **Implemented** in Header.jsx, Hero.jsx, Footer.jsx, App.jsx, and index.css.

FR-02 Expense Input

- **Description:** Allow users to edit an expense label and total amount.
- **Inputs:** Expense text up to 60 characters; decimal amount text input.
- **Expected behaviour:** The expense label falls back to Shared expense when trimmed input is empty. Amount updates calculations immediately.
- **Validation and edge cases:** Amount is required, positive, limited to two decimal places, and capped at ₹10,00,000.
- **Actual status:** **Partially implemented** because the final parser permits ₹0.01 to ₹0.99, while the original SRS specified a ₹1 minimum. All other stated behaviour is implemented.

FR-03 Participant Management

- **Description:** Allow editing, adding, and removing participants.
- **Inputs:** Two to six participant name fields, each limited to 24 characters.
- **Expected behaviour:** Add appends a new participant with a stable session ID. Remove deletes the selected participant. Add disables at six and remove disables at two.
- **Validation and edge cases:** Blank and case-insensitive duplicate trimmed names are invalid. If the removed participant is the payer, the first remaining participant becomes payer.
- **Actual status:** **Implemented** in SplitCalculator.jsx and validateParticipants.

FR-04 Payer Selection

- **Description:** Select the participant who paid the total.
- **Inputs:** Native select generated from the current participant array.
- **Expected behaviour:** Changing payer recalculates the payer label, settlement directions, and amount receivable.
- **Validation and edge cases:** Blank participant inputs use a temporary Person N option label; valid results remain suppressed until names are corrected.
- **Actual status:** **Implemented** in SplitCalculator.jsx and splitEvenly.

FR-05 Validation

- **Description:** Prevent invalid data from producing a misleading settlement.
- **Inputs:** Amount and participant values.

- **Expected behaviour:** Inline errors appear beneath the relevant field. Invalid data replaces the result with a neutral correction state.
- **Validation and edge cases:** Empty, zero, negative, over-limit, and over-precision amounts are rejected. Participant names are trimmed, required, and unique without case sensitivity. No alert dialog is used.
- **Actual status:** **Implemented** in `parseAmountToPaise`, `validateParticipants`, and `InvalidResult`.

FR-06 Split Calculation

- **Description:** Calculate equal, custom-amount, or itemised shares using integer paise.
- **Inputs:** Valid total or item amounts, ordered participants, payer ID, split method, custom shares, and item participant selections.
- **Expected behaviour:** Equal and per-item allocations distribute remainder paise deterministically. Custom shares must total the expense exactly. Assigned shares always equal the original total.
- **Validation and edge cases:** Item lines require a name, positive amount, and at least one participant. The payer falls back to the first participant if an unknown payer ID reaches the utility.
- **Actual status:** **Implemented** in `splitEvenly`, `validateCustomShares`, `calculateItemisedSplit`, and `splitWithShares`.

FR-07 Settlement Result

- **Description:** Present a clear result for a valid split.
- **Inputs:** Expense label, total, participants, payer, shares, and settlements.
- **Expected behaviour:** Show total, paid-by label, per-person share, participant count, one sentence per non-payer, ready status, and total receivable.
- **Validation and edge cases:** A share range is shown when remainder paise make participant shares differ by one paise. Long names may wrap inside settlement rows.
- **Actual status:** **Implemented** in `ValidResult` and `formatPerPersonShare`.

FR-08 Copy Result

- **Description:** Copy or share a plain-text settlement summary.
- **Inputs:** Valid calculated result.
- **Expected behaviour:** Copy the current or saved settlement through the Clipboard API, with a legacy browser fallback and visible feedback.
- **Validation and edge cases:** Failure does not affect the calculator; native Web Share is not implemented.
- **Actual status:** **Implemented** in `copyToClipboard`, `SplitCalculator`, and `HistoryPanel`.

FR-09 Local Persistence and History

- **Description:** Save and restore complete valid calculator snapshots in the current browser.
- **Inputs:** Expense, split method, amount or items, participants, custom shares, payer, settlement text, and save time.
- **Expected behaviour:** Keep the 20 most recent entries; allow restore, copy, and individual deletion after refresh.
- **Validation and edge cases:** Malformed storage safely resolves to an empty history. Storage failure does not break calculation.
- **Actual status:** **Implemented** in `history.js`, `SplitCalculator`, and `HistoryPanel`.

9. Calculation Logic

9.1 Real Algorithm

1. Trim the amount string and validate it with a decimal pattern allowing up to two decimal places.
2. Split the rupee and decimal portions.
3. Convert to integer paise without floating-point multiplication:

```
totalPaise = Number(rupeesPart) * 100  
            + Number(decimalPart padded to two digits)
```

4. Let n be the participant count.
5. Calculate:

```
baseShare = floor(totalPaise / n)  
remainder = totalPaise mod n
```

6. Assign $\text{baseShare} + 1$ paise to the first remainder participants in array order; assign baseShare to the others.
7. Treat the selected payer's assigned share as their own contribution.
8. Generate one settlement sentence for each non-payer using that participant's assigned share.
9. Calculate receivablePaise as totalPaise minus the payer's assigned share.
10. Format values with `Intl.NumberFormat` using locale `en-IN` and currency `INR`.

For a custom split, each entered share is parsed independently into paise and the result remains invalid until the exact assigned total equals the bill total. For an itemised split, every line is parsed into paise and divided only among its selected participants using the same deterministic base-share and remainder rule. Per-person item shares are accumulated before settlement generation.

9.2 Verified Default Example

Input

- Expense: Monthly groceries
- Total: ₹2,400.00
- Participants: Aaryan, Riya, Kabir, Meera
- Paid by: Aaryan

Calculation

```
240000 paise / 4 = 60000 paise each  
Remainder = 0  
Payer contribution = ₹600.00  
Payer receivable = ₹1,800.00
```

Settlement

- Riya pays Aaryan ₹600.00
- Kabir pays Aaryan ₹600.00
- Meera pays Aaryan ₹600.00

Direct execution of the production split utility confirmed four shares of 60,000 paise and exactly three settlement rows.

9.3 Verified Remainder Example

For ₹1,000.00 divided among three participants, the utility assigns 33,334 paise, 33,333 paise, and 33,333 paise. The sum is exactly 100,000 paise, or ₹1,000.00.

10. Design and UX Specification

10.1 Visual Direction

The as-built interface is a calm, crisp fintech experience with a neutral canvas, decisive navy actions, restrained mint feedback, generous whitespace, precise borders, and an emphasized result panel. Its restraint is informed by the clarity associated with Notion, Apple, and Uber without copying their layouts, assets, or trademarks.

10.2 Colour Palette

Token	Value	Actual usage
Ink	#0A2540	Headings, primary actions, footer, result panel
Mint	#3ECF8E	Success and brand accents
Mint dark	#147A50	Accessible accent text and focus-related borders
Cloud	#F6F9FC	Page and calculator section background
White	#FFFFFF	Cards, fields, and neutral surfaces
Slate	#52606D	Body and supporting text
Border	#DDE5EC	Dividers and control outlines
Error	#C73E4D	Validation borders and messages

10.3 Typography

- Font stack: Inter, ui-sans-serif, system-ui, -apple-system, BlinkMacSystemFont, Segoe UI, sans-serif.
- No remote font request is required.
- H1 uses clamp(2.8rem, 11vw, 5.8rem), 0.95 line height, and tight tracking.
- Section headings use a fluid clamp from 2.15rem to 3.8rem.
- Numeric result typography uses tabular figures and a large fluid scale.

10.4 Spacing and Layout

- Page shell maximum width: 1180 pixels.
- Mobile gutters: 20 pixels.
- Tablet gutters: 32 pixels from 760 pixels.
- Desktop gutters: 48 pixels from 900 pixels.
- Section padding: 76 pixels mobile, 96 pixels tablet, and 112 pixels desktop.
- Desktop hero and calculator become balanced two-column layouts at 900 pixels.
- Mobile calculator order remains Bill details -> People -> Paid by -> Result.

10.5 Component Styling

- Ordinary cards use white surfaces, #DDE5EC borders, 20-28 pixel radii, and subtle shadows.
- The navy result panel uses the strongest elevation and contrast.

- Inputs use 13 pixel radii and a 50 pixel minimum height; the amount field is 64 pixels.
- Primary buttons are filled navy or mint. Secondary actions use outlined or text treatments.
- Lucide icons provide a consistent familiar icon system.

10.6 Interaction States

- Hover, active, disabled, invalid, and focus-visible states are explicitly styled.
- Focus rings use a three-pixel translucent mint outline or focus-within shadow.
- Invalid fields use the error token plus a tinted background and visible text message.
- Disabled add/remove controls reduce opacity and use a not-allowed cursor.
- CSS transitions run at approximately 180 milliseconds.

10.7 Motion Behaviour

Smooth anchor scrolling and CSS transitions provide restrained motion. Under prefers-reduced-motion, smooth scrolling is disabled and animation/transition durations are reduced to 0.01 milliseconds.

10.8 Applied Interaction Laws

- **Hick's Law:** three clearly labelled split methods use progressive disclosure, equal remains the default, and the hero exposes one dominant primary action.
- **Fitts's Law:** visible interactive elements measured at 320, 390, and 1440 pixels meet the 44 by 44 pixel minimum; mobile gutters keep controls away from screen edges.
- **Jakob's Law:** familiar anchors, filled and outlined buttons, labels above fields, inline errors, native select behaviour, and conventional vertical scrolling require no explanation.

11. Accessibility Requirements

ID	Requirement	As-built evidence	Status
A11Y-01	Semantic landmarks	header, nav, main, section, footer, lists, fieldsets, and definition lists	Implemented
A11Y-02	Exactly one H1	Hero contains the single H1; live DOM check returned one	Implemented
A11Y-03	Associated form labels	All input and select IDs have matching labels; live DOM check passed	Implemented
A11Y-04	Keyboard-operable controls	Native anchors, buttons, inputs, and select are used in DOM order	Implemented
A11Y-05	Visible focus states	focus-visible and focus-within styles are defined for controls and links	Implemented
A11Y-06	Inline validation	aria-invalid, aria-describedby, role=alert, and adjacent messages are present	Implemented
A11Y-07	Live result updates	Result panel uses aria-live=polite and aria-atomic=true	Implemented
A11Y-08	Reduced motion	prefers-reduced-motion rule disables smooth and nonessential motion	Implemented
A11Y-09	Minimum touch targets	Live computed checks found no visible control below 44 by 44 pixels	Implemented
A11Y-10	Skip navigation	Skip to main content link becomes visible on focus	Implemented
A11Y-11	Colour contrast	Dark text and result surfaces use high-contrast tokens; no formal WCAG contrast report was generated	Partially implemented
A11Y-12	Screen-reader audit	Semantic and ARIA source review completed; no dedicated screen-reader session was run	Partially implemented

12. Non-Functional Requirements

NFR-01 Performance - Implemented

The Vite production build completes successfully. The page uses no remote images, video, analytics, or runtime API calls. The generated bundle is static. A formal Lighthouse score was not recorded.

NFR-02 Responsiveness - Implemented

Live computed checks at 320, 390, and 1440 pixels found document widths equal to viewport widths, no horizontal overflow, and no undersized visible controls.

NFR-03 Reliability - Implemented

Default data produces a complete result immediately. Invalid input suppresses settlement values. Native browser behaviour and deterministic utilities avoid network failure modes after load.

NFR-04 Maintainability - Implemented

The repository separates presentational components, calculator state, split utilities, and global styles. Stable participant IDs are used as React keys. No unnecessary framework layer was added.

NFR-05 Browser Compatibility - Partially implemented

The app relies on broadly supported modern React, CSS, native form controls, and Intl.NumberFormat. The live deployment was checked in headless Chrome. A formal multi-browser matrix was not executed.

NFR-06 Privacy - Implemented

No analytics, cookies, credential collection, backend submission, or persistent storage is present in the source.

NFR-07 Security Boundary - Implemented

The project makes no bank-grade or regulatory claims and exposes no secrets or privileged server surface. This is a scope boundary, not a claim that an independent security audit was performed.

NFR-08 Deployment Reliability - Implemented

npm run build produces dist, the Vercel endpoint returned HTTP 200, and no environment variables or rewrites are required for the single route.

13. Privacy and Security Scope

SPLITPE is a prototype.

- It does not process payments.
- It does not collect bank details or UPI credentials.
- It does not claim bank-grade encryption, regulatory approval, or bank status.
- It does not use authentication, a database, a backend API, analytics, or tracking.
- Active calculator state is held in React memory inside the browser.
- Saved expense snapshots are stored in browser localStorage under a versioned SPLITPE key.
- Up to 20 valid snapshots persist across refresh and can be restored or individually deleted.
- Malformed or unavailable storage safely falls back to an empty history.
- No calculator or history data is sent to a backend.

14. Technical Architecture

14.1 As-Built Stack

Layer	Package or technology	Declared version
UI library	React	^19.0.0
Browser renderer	React DOM	^19.0.0
Build tool	Vite	^6.0.5
CSS framework	Tailwind CSS	^4.0.0
Tailwind integration	@tailwindcss/vite	^4.0.0
React compiler integration	@vitejs/plugin-react	^4.3.4
Icon system	lucide-react	^0.468.0
Application language	JavaScript with JSX	ES modules
Deployment	Vercel static hosting	dist output

14.2 Architecture Flow

```
graph TD
    A[User interaction] --> B[React components and SplitCalculator state]
    B --> C["src/utils/split.js  
parsing -> validation -> equal/custom/itemised split -> INR formatting"]
    C --> D[Browser-rendered result]
    D --> E["src/utils/history.js -> browser localStorage"]
    E --> F[No backend, database, authentication, cloud sync, or payment API]
```

App.jsx composes the single page. SplitCalculator.jsx owns active state and derives the current calculation with useMemo. split.js contains pure functions for amount parsing, participant validation, deterministic split modes, settlement text, and currency presentation. history.js validates and safely reads or writes localStorage history; HistoryPanel.jsx renders its controls.

15. Component and File Structure

```
src/
  components/
    BrandMark.jsx
    Header.jsx
    Hero.jsx
    HowItWorks.jsx
    SplitCalculator.jsx
    HistoryPanel.jsx
    TrustStrip.jsx
    Footer.jsx
  utils/
    split.js
    history.js
    split.test.js
    history.test.js
  App.jsx
  main.jsx
  index.css
public/
  favicon.svg
index.html
vite.config.js
package.json
```

File	Responsibility
App.jsx	Composes the single route and semantic page landmarks
Header.jsx	Sticky primary navigation and wordmark
BrandMark.jsx	Reusable inline vector logo mark
Hero.jsx	Product promise, CTAs, and example split preview
HowItWorks.jsx	Ordered Add - Split - Settle sequence
SplitCalculator.jsx	Split-mode state, participant and item controls, clipboard actions, history snapshots, validation, and result UI
HistoryPanel.jsx	Browser-local history list with restore, copy, and delete controls
TrustStrip.jsx	Client-side and payment-boundary messaging
Footer.jsx	Final calculator CTA and minimal product footer
split.js	Amount parsing, validation, equal/custom/itemised paise logic, settlement text, and INR formatting
history.js	Versioned localStorage reads, writes, validation, ordering, and 20-entry cap
split.test.js and history.test.js	Focused Node tests for calculation and storage behaviour
index.css	Tokens, responsive layout, components, states, focus, and reduced motion
main.jsx	React DOM entry point and StrictMode wrapper
public/favicon.svg	Standalone browser icon matching the brand mark

16. Requirement Traceability Matrix

ID	Requirement summary	Priority	Status	Evidence / component	Verification method
P0-01	Product hero and primary CTA	P0	Implemented	Hero.jsx	Source and live visual inspection
P0-02	Add - Split - Settle explanation	P0	Implemented	HowItWorks.jsx	Source and live visual inspection
P0-03	Believable sample split card	P0	Implemented	Hero.jsx	Source and default values inspected
P0-04	Honest trust strip	P0	Implemented	TrustStrip.jsx	Source inspection
P0-05	Mobile-first responsiveness	P0	Implemented	index.css	Live computed checks at 320 and 390
P0-06	Single live URL	P0	Implemented	Vercel deployment	HTTP 200 check
P1-01	Live multi-mode split calculator	P1	Implemented	SplitCalculator.jsx	Source, utility tests, and browser interaction
P1-02	Instant result updates	P1	Implemented	useMemo in SplitCalculator.jsx	Source inspection
P1-03	Validation	P1	Implemented	parseAmountToPaise and validateParticipants	Direct utility execution
P1-04	Interaction polish	P1	Implemented	anchors and index.css states	Source and live visual inspection
P1-05	Accessible controls	P1	Implemented	labels, native controls, CSS focus	DOM and source checks
P2-01	Copy result	P2	Implemented	copyToClipboard and result/history actions	Clipboard-permitted browser check
P2-02	Web share	P2	Deferred	No component present	Repository search
P2-03	Local persistence	P2	Implemented	history.js and HistoryPanel.jsx	Unit test and reload check
P2-04	Reduced motion	P2	Implemented	index.css media query	Source inspection
FR-01	Navigation and section scrolling	P0	Implemented	Header.jsx, Hero.jsx, Footer.jsx	Anchor target inspection
FR-02	Expense label and amount	P1	Partially implemented	SplitCalculator.jsx, parseAmountToPaise	Source and direct utility execution
FR-03	Participant management	P1	Implemented	SplitCalculator.jsx	Source state-handler inspection
FR-04	Payer selection	P1	Implemented	SplitCalculator.jsx	Direct split execution with changed payer
FR-05	Inline validation	P1	Implemented	SplitCalculator.jsx and split.js	Direct invalid-input execution
FR-06	Integer-paise split modes	P1	Implemented	splitEvenly, validateCustomShares, calculateItemisedSplit	Unit tests
FR-07	Settlement result	P1	Implemented	ValidResult	Source and default test
FR-08	Copy result	P2	Implemented	copyToClipboard, ValidResult, HistoryPanel	Browser clipboard check
FR-09	Local persistence and history	P2	Implemented	history.js, HistoryPanel	Unit test and browser reload
A11Y-01	Semantic and labelled controls	P1	Implemented	App.jsx and SplitCalculator.jsx	DOM and source checks

ID	Requirement summary	Priority	Status	Evidence / component	Verification method
A11Y-02	Live result announcement	P1	Implemented	aria-live result panel	Source inspection
A11Y-03	Reduced motion	P2	Implemented	index.css	Source inspection
NFR-01	Production build	P0	Implemented	package.json and Vite	npm run build
NFR-02	No runtime secrets or backend	P0	Implemented	Static repository	Package and source inspection
NG-01	Real payment processing	Non-goal	Not applicable	Explicit trust copy	Source inspection
NG-02	Authentication and accounts	Non-goal	Not applicable	No dependencies or UI	Package and source inspection
NG-03	Real payment processing	Non-goal	Not applicable	No payment integration	Source inspection

17. Testing and Acceptance Criteria

Testing evidence in this final specification distinguishes direct execution, live browser checks, source verification, and items not independently verified.

Test	Expected result	Evidence	Result
Default ₹2,400 split by four	₹600.00 each and three settlement rows	Direct execution of splitEvenly	Passed
₹1,000 split by three	33,334 + 33,333 + 33,333 paise = 100,000	Direct execution of splitEvenly	Passed
Custom ₹2,400 split	Entered shares must sum to exactly 240,000 paise	Node test and live browser interaction	Passed
Itemised allocation	Each line is allocated only to its selected people and totals remain exact	Node test and live browser interaction	Passed
Payer changes to Riya	All non-payers pay Riya	Direct execution generated three updated sentences	Passed
Add and remove participant	List changes within boundaries	addParticipant and removeParticipant source inspection	Verified
Remove selected payer	First remaining participant becomes payer	removeParticipant source inspection	Verified
Blank participant name	Inline error and invalid result	Direct validateParticipants execution plus UI source	Passed
Duplicate participant names	Case-insensitive duplicate errors	Direct validateParticipants execution	Passed
Invalid amounts	Blank, zero, negative, over-limit, and over-precision rejected	Direct parseAmountToPaise execution	Passed
Minimum and maximum participants	Remove disabled at two; add disabled at six	Button conditions and handler guards inspected	Verified
Responsive widths	No horizontal overflow at 320, 390, and 1440	Live Chrome computed DOM checks	Passed
Touch targets	No visible interactive item below 44 by 44 pixels	Live Chrome computed DOM checks at three widths	Passed
Labels and H1	All controls labelled; one H1	Live DOM checks	Passed
Keyboard and focus behaviour	Native DOM order and visible focus CSS	Source inspection; full manual Tab traversal not repeated	Partially verified
Production build	Vite build completes and writes dist	npm run build on 17 July 2026	Passed
Live deployment	Production endpoint is reachable	HTTP 200 and live screenshots	Passed
Clipboard result	Valid settlement copies with browser clipboard permission	Headless Chrome permission-enabled check	Passed
Local history reload	Saved entry remains after page reload	Unit test and headless Chrome reload	Passed

Not Independently Verified

- Formal Lighthouse performance or accessibility scores.
- Dedicated screen-reader testing.
- Safari, Firefox, and Edge test matrix.
- Zoom at 200 percent.
- A maintained automated end-to-end browser test suite; focused scripted browser checks were run for this release.
- Vercel behaviour from multiple geographic regions.

These items are not represented as passed.

18. Deployment

Item	As-built value
Platform	Vercel
Framework preset	Vite
Install command	npm install
Production build command	npm run build
Output directory	dist
Application route	/
SPA rewrites	Not required; one route and native hash anchors
Environment variables	None
Backend services	None
Live application	https://split-pe-neon.vercel.app/
GitHub repository	https://github.com/Aaryan0612/SplitPe

The final production build completed successfully with Vite 6.4.3, and the deployed URL returned HTTP 200 during final documentation verification.

19. Limitations and Future Scope

The following capabilities are realistic extensions but are not part of the completed prototype:

1. **Authentication and shared households:** accounts, invitations, member roles, and cross-device access.
2. **Synced expense history:** accounts, cross-device records, edits, balances, and household timelines.
3. **Advanced split rules:** percentages, weights, quantities, receipt scanning, and per-item payer assignment.
4. **UPI settlement intent:** compliant deep links or QR handoff without misrepresenting payment processing.
5. **Reminders:** opt-in settlement prompts and status tracking.
6. **Offline/PWA support:** installability, cached shell, and queued local changes.

Any production expansion would require a separate privacy, security, data-retention, and payment-compliance specification.

20. Conclusion

SPLITPE delivers the intended internship outcome: a coherent, deployed, mobile-first fintech prototype that explains a real student problem and solves its core calculation task. The final application provides exact equal, custom, and itemised splits, copyable settlement guidance, browser-local history, robust validation, accessible interaction patterns, and honest scope boundaries without introducing unnecessary infrastructure.

The as-built result is intentionally focused. It demonstrates product thinking, interface design, responsive implementation, deterministic financial calculation, and release discipline while clearly separating delivered capability from future product scope.